

Restaurant Management System (RMS)

Technology Stack & Architecture Report

1. Overview

The Restaurant Management System (RMS) is a multi-tenant SaaS platform built to handle core restaurant operations including dashboard analytics, table management, food item management, order placement and tracking, and sales history reporting. The application enforces strict data isolation for each restaurant entity.

2. Architecture Pattern

The system follows a standard Client-Server architecture utilizing the MERN stack (MongoDB, Express.js, React, Node.js). It implements a Single Page Application (SPA) frontend that communicates with a RESTful backend API. Multi-tenancy is achieved through a "Shared Database + Tenant ID" pattern, where every data entity is scoped by a unique restaurantId.

3. Frontend Environment

Core Framework:

- React 19: The latest version of the React library for building user interfaces.

Styling & UI:

- Tailwind CSS (v2.x compat): Utility-first CSS framework for rapid and highly customizable UI development.
- Lucide-React: Icon library for modern, consistent UI iconography.
- React-Datepicker & Date-Fns: For premium, customizable date range selection components in reports.

Export & Reporting Tools:

- jsPDF & jsPDF-AutoTable: Client-side PDF generation for sales reports.
- XLSX (SheetJS): Client-side Excel document generation for data exports.

4. Backend Environment

Core Framework:

- Node.js & Express.js: Fast, unopinionated, minimalist web framework for building the RESTful API routing layer.

Security & Authentication:

- JSON Web Tokens (JWT): Stateless authentication providing secure, encoded user sessions.
- bcryptjs: Cryptographic hashing for secure password storage.
- CORS: Configured for safe Cross-Origin Resource Sharing between frontend and backend.

5. Database Layer

Database Engine:

- MongoDB: NoSQL document database providing flexible, JSON-like document structures.

ODM (Object Data Modeling):

- Mongoose: Elegant MongoDB object modeling for Node.js. It provides schema validation, business logic hooks, and query abstractions.

Key Collections:

- Users, Restaurants, FoodItems, Categories, Orders, and SalesHistory.
- Every entity utilizes the restaurantId index to strictly prevent cross-tenant data leakage.

6. API Design

The API adheres to RESTful principles. Key features include:

- Protected Endpoints: All operational routes pass through a custom authentication middleware that verifies the JWT and automatically enforces the restaurantId scope on all subsequent database queries.
- Dynamic Aggregations: Endpoints like the dashboard and sales history leverage powerful MongoDB aggregation pipelines to filter and group data dynamically based on user-selected date ranges.
- Transactional Safety: Operations (e.g. signup) are atomic, with automatic rollbacks applied if subsequent entity creations fail.

Generated by Agency Document Generator Agent